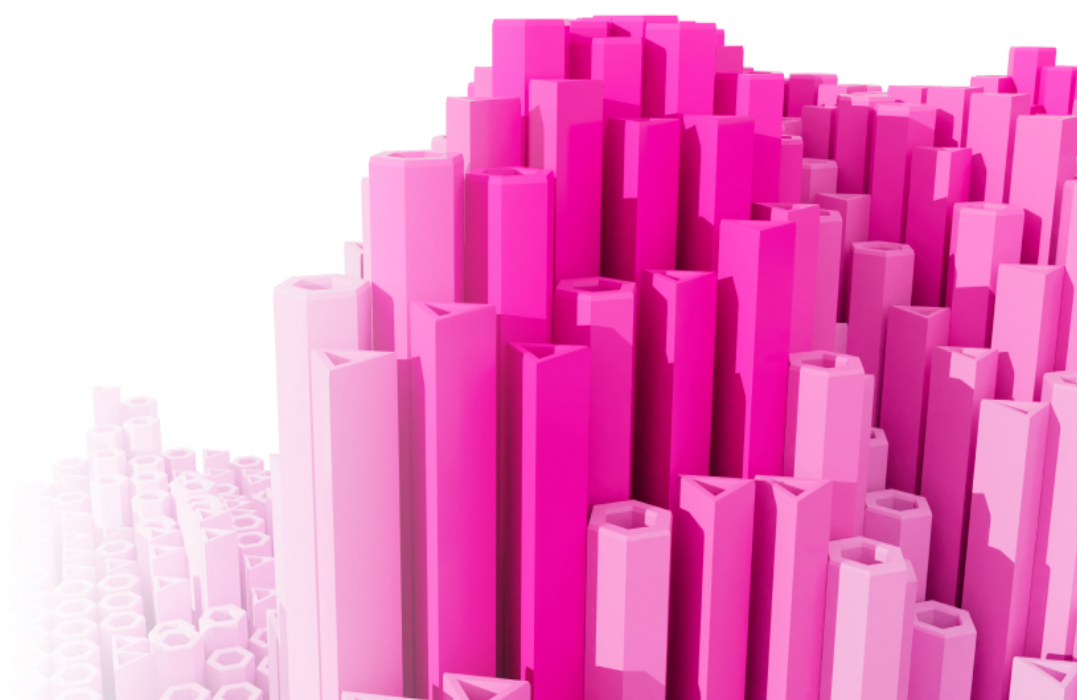


# Zeta Chain Penetration Test

Web Application Penetration Test

June 02, 2025



# Table of Contents

---

|                                               |           |
|-----------------------------------------------|-----------|
| <b>Executive Summary</b>                      | <b>3</b>  |
| <b>Technical Summary</b>                      | <b>4</b>  |
| Addressing the Highest Severity Vulnerability | 5         |
| Addressing the Most Prevalent Vulnerability   | 6         |
| Vulnerabilities by Severity                   | 7         |
| Vulnerabilities by Asset                      | 8         |
| Vulnerabilities by Category                   | 9         |
| <b>Vulnerability Details</b>                  | <b>10</b> |
| <b>Methodology</b>                            | <b>13</b> |
| Statement of Coverage                         | 13        |
| Summary of Assets                             | 14        |
| HackerOne Security Checklists                 | 15        |
| Vulnerability Classification & Severity       | 17        |
| Approach                                      | 18        |
| Testing Team                                  | 21        |
| Tools                                         | 22        |
| Restrictions                                  | 22        |
| <b>Disclaimer</b>                             | <b>23</b> |

# Executive Summary

ZetaChain engaged HackerOne to perform a HackerOne penetration test from May 09, 2025 to May 23, 2025. The scope of testing included three (3) web applications. This report reflects a point-in-time record of the state of security across applications and systems tested during this period.

The security assessment was conducted using a community-driven pentesting methodology, which draws on the [National Institute of Standards and Technology \(NIST\)](#) and [Open Web App Security Project \(OWASP\)](#) frameworks. From its community of over one million hackers, HackerOne curated a set of top-tier researchers to focus on identifying vulnerabilities in targets defined by ZetaChain to be in-scope during the agreed-upon testing window while abiding by the policies set forth by ZetaChain. The [Approach](#) section contains additional information related to the testing methodology used in this engagement.

During this timeframe, the pentest team identified the following vulnerabilities:

| Critical                                                                            | High                                                                                | Medium                                                                              | Low                                                                                 | None                                                                                  | Total |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------|
|  |  |  |  |  |       |
| 0                                                                                   | 0                                                                                   | 0                                                                                   | 0                                                                                   | 1                                                                                     | 1     |

Table: Severity of findings

# Technical Summary

---

During the engagement, the pentesters found one (1) unique vulnerabilities across one (1) different **Common Weakness Enumeration (CWE)** and **Common Attack Pattern (CAPEC)** categories. The most common vulnerability type was Information Disclosure with one (1) unique vulnerability. Reproduction steps for each vulnerability can be viewed and exported via the program inbox in the **HackerOne Portal**.

In this penetration testing assessment, three web applications associated with ZetaChain were assessed the blockchain explorer, main website, and ecosystem website. The testing was conducted using a black-box approach, with testers able to use MetaMask test accounts. The primary focus was on identifying vulnerabilities and ensuring robust security measures to protect both investors and customers.

The assessment revealed one finding: an exposed README.md file on the main website (www.zetachain.com). This information disclosure issue, while rated as having no direct security impact, could potentially provide unnecessary information to attackers about the site's structure or technologies used. Despite this minor finding, the overall security posture of the assessed assets appears to be relatively strong, given the lack of more severe vulnerabilities discovered.

# Addressing the Highest Severity Vulnerability

Based on the provided information, there were no severe vulnerabilities reported during this engagement. The only finding mentioned is an “Exposed README.md File” with a severity rating of “none” on the asset “<https://www.zetachain.com/>”. This type of issue is generally considered low risk and is not typically classified as a severe vulnerability.

An exposed README.md file may contain sensitive information about the website’s structure, dependencies, or deployment processes. While this is not ideal from a security perspective, it does not directly pose a significant threat to the system’s integrity or user data.

It’s important to note that while this finding is relatively minor, it’s always a good practice to minimize unnecessary information exposure to potential attackers.

To address this finding, implement the following:

It is recommended to remove or restrict access to the README.md file on the production server. This file should be kept in the development environment and version control system, but not deployed to the public-facing website.

See the following for more information:

- [PortSwigger: Information Disclosure Vulnerabilities](#)
- [OWASP: A3:2017-Sensitive Data Exposure](#)

# Addressing the Most Prevalent Vulnerability

Based on the provided information, there is only one finding reported - "Exposed README.md File" with CWE-200 (Information Exposure). This vulnerability was found on the main website asset (<https://www.zetachain.com/>).

An exposed README.md file can potentially reveal sensitive information about the website's structure, configuration, or development process. This type of information disclosure could be exploited by attackers to gain insights into the system and potentially find other vulnerabilities.

To address this finding, implement the following:

- Remove or restrict access to README.md files on production servers.
- Implement proper access controls to prevent unauthorized access to sensitive files.
- Use configuration management tools to ensure that development files are not deployed to production environments.
- Regularly audit web server configurations and file permissions to identify and remove any exposed sensitive files.

See the following for more information:

- [PortSwigger: Information Disclosure Vulnerabilities](#)
- [OWASP: A3:2017-Sensitive Data Exposure](#)

# Vulnerabilities by Severity

The following table documents each vulnerability identified by the testers:

| Report ID | Vulnerability          | Severity | CVSS | Reference | Status |
|-----------|------------------------|----------|------|-----------|--------|
| #3137276  | Exposed README.md File | None     | 0.0  | CWE-200   | Open   |

Table: Vulnerabilities by severity

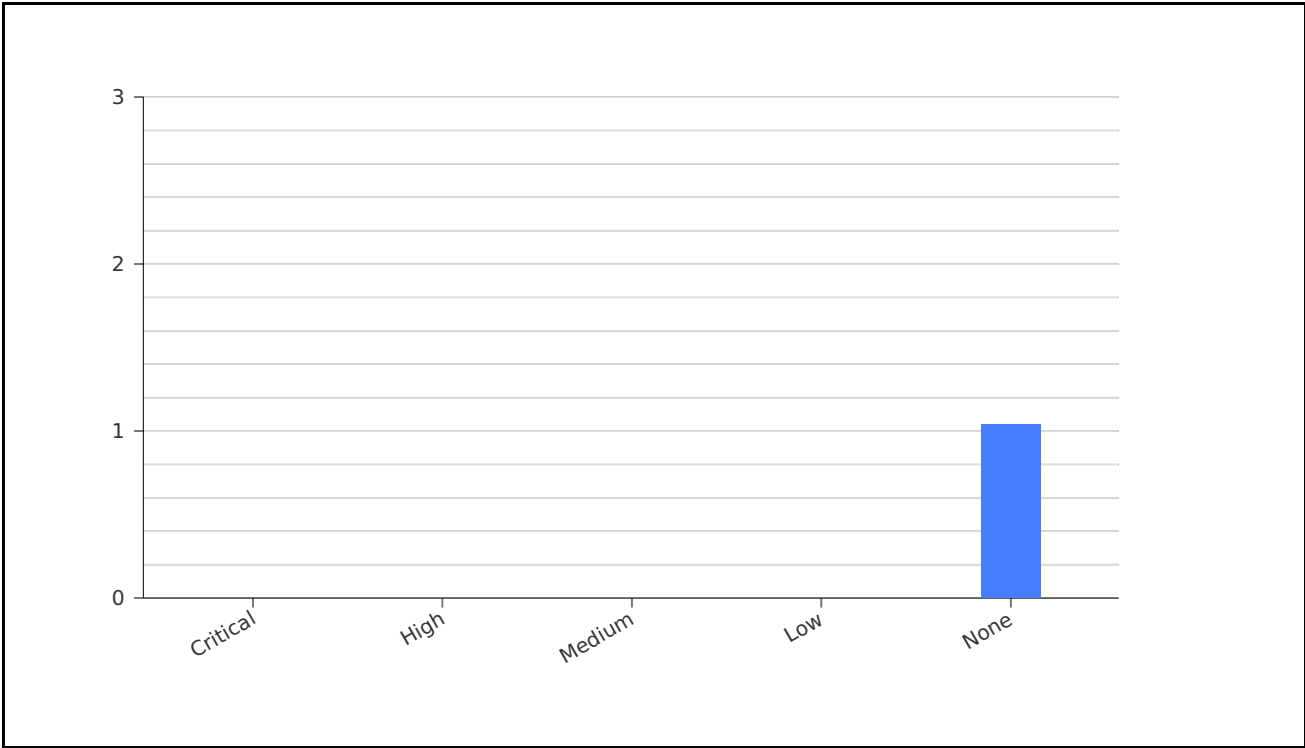
# Vulnerabilities by Asset

Pentesters reported vulnerabilities in the following assets:

| Asset                          | Critical                                                                          | High                                                                              | Medium                                                                             | Low                                                                                 | None                                                                                | Total |
|--------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------|
|                                |  |  |  |  |  |       |
| https://www.zetachain.com/     | -                                                                                 | -                                                                                 | -                                                                                  | -                                                                                   | 1                                                                                   | 1     |
| https://hub.zetachain.com      | -                                                                                 | -                                                                                 | -                                                                                  | -                                                                                   | -                                                                                   | -     |
| https://explorer.zetachain.com | -                                                                                 | -                                                                                 | -                                                                                  | -                                                                                   | -                                                                                   | -     |
| Total                          | 0                                                                                 | 0                                                                                 | 0                                                                                  | 0                                                                                   | 1                                                                                   | 1     |

Table: Severity of findings by asset

The following graph presents the number of identified vulnerabilities and their respective severity ratings:



Graph: Number of vulnerabilities per severity



# Vulnerabilities by Category

The following tables list the findings further divided by their vulnerability types, as defined by weakness or attack pattern category, and their distribution of severity:

| Category               | Critical                                                                          | High                                                                              | Medium                                                                            | Low                                                                                 | None                                                                                | Total |
|------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------|
|                        |  |  |  |  |  |       |
| Information Disclosure | -                                                                                 | -                                                                                 | -                                                                                 | -                                                                                   | 1                                                                                   | 1     |

Table: Severity of finding(s) by weakness (CWE)

See the [Vulnerability Classification & Severity](#) section of this report for a breakdown of severities and their mapping to Common Vulnerability Scoring System (CVSS) scores.

# Vulnerability Details

## #3137276 Exposed README.md File

### Affected Asset

- `https://www.zetachain.com/`

**Severity: None (0.0)**

### Vulnerability Overview

Information Disclosure vulnerabilities occur when a web application unintentionally leaks sensitive information to its users. Depending on the amount and type of data leaked, this could have consequential reputational damage, punitive regulatory fines and/or loss of investor and customer confidence.

### Finding Details

The web app has a README.md file in web server public root, allowing it to be downloaded by anyone accessing that file path.

The README.md file can reveal sensitive information and is not needed from a functional perspective.

### Steps To Reproduce

1. Visit <https://www.zetachain.com/README.md>
2. Observe that the file gets downloaded

```
File Edit Search View Document Help
+ [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons]
1 # ZetaChain Apps
2
3 ## Pre-requisites
4
5 - Vercel CLI
6
7 ```bash
8 npm i -g vercel
9 ```
10
11 - Yarn
12
13 ```bash
14 npm i -g yarn
15 ```
16
17 - Doppler CLI
18
19 https://docs.doppler.com/docs/install-cli
20
21 ## Setting up your local environment
22
23 ### Environment variables
```

## Recommendations

The following are some general best practices that you can follow to minimize the risk of information disclosures:

- Make sure that everyone involved in producing the website is fully aware of what information is considered sensitive. Sometimes seemingly harmless information can be much more useful to an attacker than people realize. Highlighting these dangers can help make sure that sensitive information is handled more securely in general by your organization.
- Audit any code for potential information disclosure as part of your QA or build processes. It should be relatively easy to automate some of the associated tasks, such as stripping developer comments.
- Make sure you fully understand the configuration settings, and security implications, of any third-party technology that you implement. Take the time to investigate and disable any features and settings that you don't actually need.

## References

See the following for more information:

- [PortSwigger: Information Disclosure Vulnerabilities](#)

- OWASP: A3:2017-Sensitive Data Exposure

## Impact

Exposes default Vercel read me file, which can be useful for attackers targeting Vercel apps.

# Methodology

## Statement of Coverage

The following table shows the scope of the assessment:

| Assets                         |
|--------------------------------|
| https://explorer.zetachain.com |
| https://www.zetachain.com/     |
| https://hub.zetachain.com      |

Table: Assessment Scope

## Summary of Assets

Based on the pentest description, <https://hub.zetachain.com> is the ecosystem website where the most important potential risks are located, particularly the XP claim feature. No vulnerabilities were identified during the assessment, indicating that the application demonstrates a strong security posture. This reflects a high level of attention to security best practices and secure development processes.

Based on the limited information provided, <https://www.zetachain.com/> appears to have minor security concerns, with only one low-severity finding related to an exposed README.md file.

The application did not exhibit any exploitable vulnerabilities during the assessment. This suggests effective implementation of security controls and a mature security posture.

The team identified all applicable test cases and respective vulnerabilities against each OWASP category, and full finding reports are available on the [HackerOne Portal](#).

# HackerOne Security Checklists

## HackerOne Web Security Checklist

| Check Name                                 | Description                                                                                                                                                                                                                                                                              |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Broken Access Control                      | Broken access control occurs when an application lacks thorough permission or role checks. These missing checks typically lead to unauthorized information disclosure, modification or destruction of data, or performing a business function outside of the typical limits of the user. |
| Cryptographic Failures                     | Sensitive data exposure relates to issues regarding insecure or missing cryptographic protocols. This may affect data transmission, storage, or access.                                                                                                                                  |
| Identification and Authentication Failures | Identification and Authentication Failures occur when an application lacks controls around the authentication process or session management. This may lead to an attacker being able to compromise an individual account or even a system-wide authentication bypass.                    |
| Injection                                  | Injection flaws occur when unvalidated/unfiltered user-supplied data is used directly by an application. Injection vulnerabilities are often found in SQL queries, OS commands, XML parsers, SMTP headers, expression languages, and ORM queries.                                        |
| Insecure Design                            | Insecure Design relates to flaws in threat modeling, secure design patterns and principles, and reference architectures. This is a broad category representing different weaknesses, expressed as "missing or ineffective control design."                                               |

| Check Name                               | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Security Logging and Monitoring Failures | Security Logging and Monitoring Failures occur when there are missing or inadequate mechanisms to help detect, escalate, and respond to active breaches. Without logging and monitoring, breaches cannot be detected.                                                                                                                                                                                                                                                                                                                                                                                         |
| Security Misconfiguration                | Security misconfiguration occurs when application development or deployment does not follow security best practices. Security misconfiguration can happen at any level of an application stack, including the network services, platform, web server, application server, database, frameworks, custom code, and pre-installed virtual machines, containers, or storage.                                                                                                                                                                                                                                      |
| Server-Side Request Forgery (SSRF)       | SSRF flaws occur whenever a web application is fetching a remote resource without validating the user-supplied URL. It allows an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall, VPN, or another type of network access control list (ACL).                                                                                                                                                                                                                                                                                     |
| Software and Data Integrity Failures     | Software and data integrity failures relate to code and infrastructure that does not protect against integrity violations. An example of this is where an application relies upon plugins, libraries, or modules from untrusted sources, repositories, and content delivery networks (CDNs). An insecure CI/CD pipeline can introduce the potential for unauthorized access, malicious code, or system compromise. Lastly, many applications now include auto-update functionality, where updates are downloaded without sufficient integrity verification and applied to the previously trusted application. |
| Vulnerable and Outdated Components       | Vulnerable and Outdated Components occurs when application developers or deployers fail to keep third-party libraries/tools up to date. This can result in cases where known vulnerabilities and their exploits are able to apply outside of their original context.                                                                                                                                                                                                                                                                                                                                          |



# Vulnerability Classification & Severity

To categorize vulnerabilities into a common vulnerability taxonomy, HackerOne uses the industry-standard [Common Weakness Enumeration \(CWE\)](#). CWE is a community-developed taxonomy of common software security weaknesses. It serves as a common language, a measuring stick for software security tools, and as a baseline for weakness identification, mitigation, and prevention efforts.

To rate the severity of vulnerabilities, HackerOne uses the industry standard [Common Vulnerability Scoring System \(CVSS\)](#) to calculate severity for each identified security vulnerability. CVSS provides a way to capture the principal characteristics of a vulnerability, and produce a numerical score reflecting its severity, as well as a textual representation of that score.

To help prioritize vulnerabilities and assist vulnerability management processes, HackerOne translates the numerical CVSS rating to five qualitative representations, Critical, High, Medium, Low, and Informational:

|                                                                                     | Severity        | CVSS Rating |
|-------------------------------------------------------------------------------------|-----------------|-------------|
|   | <b>Critical</b> | 9.0 – 10.0  |
|  | <b>High</b>     | 7.0 – 8.9   |
|  | <b>Medium</b>   | 4.0 – 6.9   |
|  | <b>Low</b>      | 0.1 – 3.9   |

**Note:** Reports marked as [None](#) are considered Informational and typically include industry best practices. They have a CVSS score of 0.0.

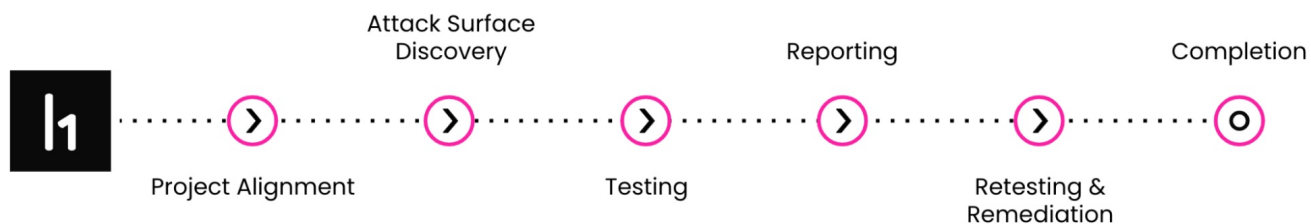
For more information about CWE, see the [MITRE CWE Project](#).  
For more information about CVSS, see the [Forum for Incident Response and Security Teams CVSS Project](#).

# Approach

A HackerOne pentest engagement follows a series of methodologies, checklists, and guidelines to ensure a balance between consistent customer experience, coverage of testing, and depth of testing. This engagement included testing to evaluate the assets for vulnerabilities included in the [SANS Top 25](#) and [OWASP Top 10](#). Using this combination of best practices and proprietary experience, HackerOne penetration tests provide a thorough level of security assurance and an unbiased assessment of the state of security for its customers.

HackerOne developed the pentesting process using industry best practices, such as [OSSTMM](#), [OWASP](#), [NIST](#), [PTES](#), and [ISSAF](#), as well as proprietary knowledge gained through HackerOne's platform, which services hundreds of ongoing or timeboxed engagements and a community of over one million hackers. Using this combination of best practices and proprietary experience, HackerOne's penetration tests provide a thorough level of security assurance and an unbiased assessment of the state of security for its customers.

## Engagement Phases



### Project Alignment

HackerOne leverages experts from several internal teams to support customers and understand the goals and expectations for the pentest engagement. HackerOne then works with our community to select highly talented and qualified pentesters that best fit the individual customer's needs and technologies.

HackerOne also works with the customer to define Rules of Engagement (RoE) for testing activities where applicable, and to establish lines of communication for all stakeholders to minimize risks to the in-scope assets. The outcome of this phase is a fully aligned team, from customer to hackers, to ensure that the engagement launches with the highest chance of success.

### Attack Surface Discovery

The selected pentesters begin testing by conducting initial discovery efforts, including verifying that hosts are alive and stable, understanding all possible functionalities, and identifying access levels on the in-scope assets. The pentesters then begin active testing activities to understand the state of perimeter defenses and inventory likely attack vectors for the environment. They will also look at core functionality to begin identifying initial weaknesses in the system.

During this phase, the team focuses on familiarizing themselves with the environment and conducting initial research towards the customer's in-scope assets, although they also report any findings that come up. Once the Pentest Team is familiar with the assets and environment that they are testing against or within, they prepare to identify likely attack vectors, and gain a deeper understanding towards the state of security for the assets or environment being tested.

### Testing

In this phase, HackerOne empowers the Pentest Team with high-level coverage requirements and detailed testing guides to ensure depth of coverage for the in-scope assets. Pentesters also launch their most aggressive attacks (within the scope of the RoE) against in-scope assets to ensure the most thorough level of security assurance for the customer. The outcome of this phase is gaining an

appropriate level of understanding about the security assurance for all assets.

## Reporting

During the Reporting phase, HackerOne collaborates with the Pentest Team and the customer to ensure that all testing efforts and details towards findings are accurately gathered and presented to the customer in deliverables. HackerOne's reports are an impartial reflection of the assessment conducted against the customer's assets during the testing timeframe. While the reports may be customized, they can not be influenced by the customer's directive. The goal of this phase is to capture the current state of security for the assets in scope, from HackerOne's perspective, in a transferable and reusable format.

## Retesting & Remediation

ZetaChain has 30 days from the last day of testing to engage HackerOne in a free retest of the findings. These retests are delivered by the original testers and are usually validated within 72 hours. This [retest window](#) ends on June 22, 2025. Any retests beyond this window will require a credit card provisioned against the program via [ZetaChain H1P Credit Card Settings](#).

# Testing Team

HackerOne creates a team from our community of over one million pentesters during the Project Alignment phase of the pentest. The [HackerOne's Clear program](#) plays a crucial role in enhancing our pentest business and Bug Bounty programs, significantly advancing HackerOne's trust revolution. Clear is essential for sourcing, ensuring engagement with verified and trustworthy professionals.

Further information on the program is available via the [HackerOne Clear](#) page.

## Testing Lead

The Testing Lead, Andreea Cristina Velicu ( [@adruga](#) ), is the primary contact for the pentesting team. It is their responsibility to ensure smooth running of testing efforts, to ensure that the team provides coverage across the assets, to review any draft reports, and to maintain communication with the client.

## Testing Support

The following pentesters supported the Testing Lead:

- - [@letstryharder](#)

## Technical Engagement Manager

[Narendra Sharma](#) is the Technical Engagement Manager for this assessment and is responsible for orchestration and quality assurance.

# Tools

During the assessment, the pentest team used a variety of tools, including:

- Nmap: A widely-used port scanning and network exploration tool, used for identifying open ports, services, and potential attack vectors.
- Burp Suite: A comprehensive web application security testing suite, employed for analyzing HTTP/HTTPS traffic, identifying vulnerabilities, and conducting manual and automated testing.
- SQLmap: A powerful SQL injection testing tool, used for detecting and exploiting SQL injection vulnerabilities in web applications.
- Nikto: A web server scanner, used for identifying misconfigurations, outdated software versions, and other potential vulnerabilities.
- SSLScan: A tool for analyzing the SSL/TLS configuration and ciphers of web servers, helping to identify weak or insecure configurations.
- Metasploit: A widely-used penetration testing framework, utilized for exploiting vulnerabilities and conducting post-exploitation activities.
- Custom scripts and tools: In addition to off-the-shelf tools, custom scripts and utilities were developed and utilized as needed to address specific testing requirements or automate certain tasks.

The selection and use of tools were tailored to the specific requirements of the engagement, the nature of the target systems, and the types of vulnerabilities being investigated.

## Restrictions

During the engagement timeframe, the team encountered some restrictions that affected the overall outcome. These restrictions included:

The assessment was conducted using black-box testing methodology for all three web applications in scope. The pentesters were also allowed to create their own Metamask account for testing purposes. Additionally, expertise in web3 and blockchain technology was required for this engagement, which may have influenced the testing approach and scope.

# Disclaimer

---

Matters raised in this report only reflect findings identified during the review, and may not represent a comprehensive statement of all weaknesses that exist, or all actions that can be taken. This work was performed under limitations of time and scope, which may not be a limitation faced by a persistent actor. The review was based at a specific point in time, in an environment where both the systems and the threat profiles are dynamically evolving. It is therefore possible that vulnerabilities exist or will arise that were not identified during the review and there may or will have been events, developments, and changes in circumstances subsequent to its issue.